

Backend Engineering Intern Assignment

Timbre Mini Processing API

Objective

Build a small backend service that demonstrates your ability to work with **file uploads, REST APIs, PostgreSQL, asynchronous processing, validation, and error handling**.

We care more about **how you structure the backend, handle failure cases, and explain your decisions** than how many features you implement.

Expected effort: 4-6 hours. Please do not over-engineer the solution.

Assignment Description

Create a backend API that allows a user to upload an audio/video asset, start a simulated processing operation, and track its status.

1. Upload a media asset

Create:

```
POST /assets
Content-Type: multipart/form-data
```

The request should contain a media file, for example:

```
file: interview.mp4
```

The API should:

- Accept an **audio or video file** using `multipart/form-data`.
- Reject requests where no file is provided.
- Validate supported file types and apply a reasonable file-size limit.
- Save the uploaded file locally.
- Generate a safe stored filename/path instead of trusting the original filename.
- Store asset metadata in PostgreSQL, including the original name, MIME type, size, storage path, and creation time.
- Return the created asset.

You may use **Multer or any equivalent approach** for multipart uploads.

2. Retrieve an asset

Create:

```
GET /assets/:assetId
```

Return the stored asset metadata. Handle invalid IDs and assets that do not exist.

3. Start a processing job

Create:

```
POST /assets/:assetId/process
Content-Type: application/json
```

Example request:

```
{
  "operation": "transcription"
}
```

Support at least:

- transcription
- noise_reduction

The API should:

1. Validate that the asset exists and the operation is supported.
2. Create a processing job with status `queued`.
3. Return the job response **without waiting for processing to finish**.
4. Process the job asynchronously and update its status:

```
queued -> processing -> completed
```

The actual media processing should be **simulated**. Real transcription or noise reduction is not required.

4. Check processing status

Create:

```
GET /jobs/:jobId
```

Return the job details, including:

- Job ID
- Asset ID
- Operation
- Status
- Created time
- Completed time, when applicable

Handle invalid and missing job IDs correctly.

5. List jobs for an asset

Create:

```
GET /assets/:assetId/jobs
```

Return all processing jobs created for that asset.

Technical Requirements

- Use **Node.js** and **Express.js**.
- Use **PostgreSQL** for persistent application data.
- You may use any PostgreSQL client or ORM.
- Handle uploads using `multipart/form-data`.
- Store uploaded files locally. Cloud storage is not required.
- Keep configuration such as database credentials in environment variables.
- Include migrations or another reproducible database setup mechanism.
- Use appropriate HTTP status codes and meaningful error responses.

- Validate request input and handle expected failures without crashing the server.
- Keep asset, job-processing, and API concerns reasonably separated.
- Simulated processing must not block the HTTP request until the work finishes.
- Include clear setup and run instructions in a README.

The solution should be straightforward for another developer to run and review.

Bonus - Not Required

Any of the following can be added if you have time, but none are required:

- Extract media metadata such as duration using `ffprobe`.
 - Add unit or integration tests for important flows.
 - Add Docker / Docker Compose for the API and PostgreSQL.
 - Store files in S3, Azure Blob Storage, or another object store.
 - Use a proper background queue instead of in-process simulated processing.
 - Add idempotency to avoid duplicate processing jobs.
 - Add retry handling for failed jobs.
 - Explain or demonstrate how you would handle two identical processing requests arriving concurrently.
-

Deliverables

Before the interview, please share:

1. **Source code** - GitHub repository preferred, or a ZIP archive.
2. **README** - setup instructions, required environment variables, database setup, how to run the API, and any assumptions/trade-offs.
3. **Database setup** - migrations, schema files, or another reproducible setup method.
4. **API documentation** - Postman, Bruno, OpenAPI/Swagger, or clear request examples in the README.

No frontend, deployed environment, or cloud account is required.

Next Steps

After submission, you will have a **30-45 minute technical discussion** covering your implementation.

Topics may include:

- Project structure and design decisions.
- Database schema and relationships.
- File-upload handling and validation.
- How asynchronous processing works in your solution.
- Error handling and failure scenarios.
- What would change if processing took 10 minutes instead of a few seconds.
- How you would handle many jobs arriving at the same time.
- How you would prevent duplicate processing.
- How you would move uploaded files from local storage to cloud object storage.

You may also be asked to explain or make a small change to your implementation during the discussion.

Evaluation Criteria

- Backend fundamentals and REST API design.
- PostgreSQL and data-modeling fundamentals.
- File-upload handling.
- Understanding of asynchronous programming.
- Input validation and error handling.
- Code quality and project structure.
- Ability to reason about edge cases and failures.
- Ability to explain technical decisions clearly.
- Ability to understand and modify your own implementation.

Use of AI tools is allowed, but **you must understand and be able to explain all code you submit**. Code that you cannot explain will be treated as a significant negative signal.